

R3 ARCHITECTURE AND DESIGN: FALCON-512 (FN-DSA) AUTHENTICATION FOR SOME/IP SERVICE DISCOVERY

Omar MAHMOUD

Purpose of This Document

This document is the design deliverable that opens the third phase (R3) of the research. It does not present new experimental results; instead it translates the conclusion reached at the end of R2 — that FN-DSA (Falcon-512) should replace ML-DSA-44 because it avoids the MTU collision entirely rather than requiring fragmentation — into a concrete protocol design that can be implemented and benchmarked on the existing someipy testbed. Everything below is a proposal to be reviewed before implementation starts, not a finished architecture.

1. Recap and Pivot Rationale

R1 identified the core problem: SOME/IP-SD has no cryptographic authentication, and the NIST post-quantum signature algorithms are all substantially larger than the 1,500-byte automotive Ethernet MTU, which rules out simply attaching a signature to an existing packet. R1's original proposal was to solve this by fragmenting a Dilithium/ML-DSA-44 signature (2,420 bytes) across two SOME/IP-SD messages (the Offer and its Repetition, ~10ms apart).

R2 built the testbed, empirically proved the routing-hijack vulnerability (a fast attacker captured the dashboard's video feed roughly 66% of the time), and benchmarked both candidate algorithms directly. ML-DSA-44 was confirmed fast (~0.25–0.27ms combined sign+verify on Apple M4/ARM64) but its signature is still 2,420 bytes. Falcon-512 was slower to sign (~2.43ms on the same hardware) but its signature is roughly 4x smaller. R2's conclusion was that this size difference matters more than the timing difference, because the MTU is a hard physical ceiling and the 50ms latency budget is not — there is still headroom left even with the slower algorithm.

This document proceeds on that basis: R3 designs a Falcon-512 authentication extension for SOME/IP-SD that fits inside a single, unfragmented UDP datagram.

2. Objectives for R3

- Define a new SOME/IP-SD Option that carries a Falcon-512 signature without breaking AUTOSAR-compliant parsing of existing options.
- Close a gap that neither R1 nor R2 addressed explicitly: what exact bytes get signed. A signature over the wrong scope can be replayed or re-attached to a different endpoint, which would defeat the whole point of authenticating Service Discovery.
- Decide how public keys reach the verifier without reintroducing a handshake (which R2's DTLS benchmark showed is too slow) or a single point of failure (which the literature review flagged for DNS-based designs).
- Specify how the someipyd gateway daemon and the ECU application scripts (camera_service.py, dashboard_client.py) need to change to verify signatures and reject forged offers.
- Produce a concrete, falsifiable experimental plan so R3's implementation phase has clear pass/fail criteria, re-using the attacker_service.py hijack scenario from R2 as the test case.

3. Algorithm Finalization: FN-DSA (Falcon-512)

Falcon has since been formally taken up by NIST as FN-DSA under FIPS 206. As of mid-2025 NIST circulated the FIPS 206 draft for public review (targeting a final standard in late 2026 or 2027), so it should still be treated as a standard in progress rather than a finalized one — a caveat worth stating explicitly in R3, since it affects how "production-ready" the design can be claimed to be.

Falcon-512 signatures are not naturally fixed-length: the reference construction produces a variable-length compressed signature and reaches a fixed maximum only if the implementation pads it. The recommended fixed ("CT",

constant-time) encoding pads every signature to 666 bytes specifically so that transmission and verification do not leak timing information about the message being signed. This project adopts the 666-byte fixed-length CT encoding for two reasons: it removes a timing side-channel, and it makes the new SOME/IP-SD Option's size predictable, which simplifies both the MTU budget calculation below and the gateway's parsing logic.

Property	ML-DSA-44 (R1/R2 candidate)	Falcon-512 / FN-DSA (R3 candidate)
Public key size	1,312 bytes	897 bytes
Signature size	2,420 bytes	666 bytes (fixed, CT-padded)
Sign time (Apple M4, measured)	~0.265 ms	~2.43 ms
Verify time (Apple M4, measured)	<0.001 ms	not yet measured — see §11
Fits inside 1,500-byte Ethernet MTU alone?	No — requires fragmentation	Yes, with room to spare

The verify-time gap in the table is intentional: R2's `crypto_test.py` already benchmarks Falcon-512 verification, but that number was reported only as console output and was not written to `R2_DATA.txt`, so it is not part of the retained dataset. Re-running and logging it is listed as the first task in the experimental plan (§11).

4. MTU Budget: Why Falcon-512 Avoids Fragmentation

R1 and R2 both state the MTU collision qualitatively. This section works the arithmetic so the claim is verifiable rather than asserted.

A standard Ethernet frame carries at most 1,500 bytes of payload after the Ethernet header. Once IPv4 (20 bytes) and UDP (8 bytes) headers are subtracted, approximately 1,472 bytes remain for the SOME/IP-SD message itself (header, entries array, and options array combined).

The AUTOSAR generic Option header used throughout SOME/IP-SD is: a 2-byte Length field, followed by a 1-byte Type field, followed by a 1-byte Reserved field, followed by the type-specific payload; the Length field's value counts everything after itself (Type + Reserved + payload). A new “PQ-Signature Option” carrying a 1-byte algorithm identifier and a 666-byte Falcon-512 signature therefore has a Length value of 1 (Type) + 1 (Reserved) + 1 (AlgID) + 666 (signature) = 669, for a total on-wire size of 671 bytes (2-byte Length field + 669).

Against the ~1,472-byte budget, 671 bytes leaves roughly 800 bytes of headroom for the rest of the message — the 16-byte SOME/IP-SD header, a single OfferService entry (16 bytes), and an IPv4 Endpoint Option (11 bytes) together account for well under 50 bytes, so the whole authenticated Offer message fits in a single, unfragmented datagram with margin to spare.

Repeating the same arithmetic for ML-DSA-44 confirms why R1 needed fragmentation in the first place: a signature option carrying a 2,420-byte signature would have an on-wire size of roughly 2,425 bytes — already larger than the entire 1,472-byte budget before anything else is added, hence R1's proposal to split it into two ~1.2KB halves sent 10ms apart across the Offer and its Repetition.

5. The PQ-Signature Option (New TLV Definition)

The design reuses the existing SOME/IP-SD Options Array mechanism rather than inventing a new message type, following the same approach R1 originally proposed. A new Option Type value is defined for the Falcon-512 signature; AUTOSAR does not publish a formally reserved “private use” range in the base specification, so the exact Type byte (proposed: 0x50) will need to be chosen to avoid collision with the standard types already assigned (0x01 Configuration, 0x04/0x06 IPv4/IPv6 Endpoint, 0x14/0x16 Multicast, 0x24 Load Balancing, and so on) and should be documented as a project-local convention, consistent with how someip already assigns its own identifiers for this testbed.

Field	Size	Description
Length	2 bytes	Value = 1 (Type) + 1 (Reserved) + 1 (AlgID) + signature length, per the

		standard Option Length convention
Type	1 byte	0x50 — proposed project-local identifier for “PQ-Signature Option”
Reserved	1 byte	0x00, per the standard convention for all existing Option types
AlgID	1 byte	0x01 = Falcon-512 / FN-DSA. Reserved for future algorithm agility (e.g. a later ML-DSA fallback).
Signature	666 bytes (fixed)	Falcon-512 signature in the constant-time (CT) padded encoding, computed over the scope defined in §6

5.1 Backward Compatibility and a Silent-Degradation Risk

The AUTOSAR SD specification requires that an entry referencing an unrecognized option simply ignore that option (SWS_SD_00661) rather than rejecting the whole message. This is convenient for incremental rollout — a PQ-Signature Option can be added to real ECUs without breaking any legacy receiver that does not understand it — but it is also a silent-degradation risk: a legacy or downgraded gateway will happily accept an unsigned or unverified Offer as if the signature had never been added, because it simply never looks at it. The design therefore needs an explicit gateway policy flag (§8/§9) that distinguishes “signature-aware and enforcing” from “signature-aware but not yet enforcing,” so this transition state is a conscious configuration choice rather than an invisible gap.

6. Signing Scope: Closing the Endpoint-Substitution Gap

This is the most important design decision in this document, and it is a gap that neither R1 nor R2 addressed. If the Falcon-512 signature only covers the OfferService Entry (Service ID, Instance ID, Major Version, TTL), an attacker does not need to forge a signature at all to hijack the route: they can simply capture one legitimate, validly-signed Offer message from the real Camera ECU, strip out the camera's IPv4 Endpoint Option, and re-broadcast the same message with their own IPv4 Endpoint Option pointing at the attacker's own port — the signature over the Entry is still perfectly valid, because the Entry itself was never modified. This is a direct continuation of the R2 hijack: instead of forging a new Offer, the attacker replays a real one with a swapped endpoint.

The fix is to define the signed payload to include the endpoint binding, not just the entry: `signature = Sign(secret_key, ServiceID || InstanceID || MajorVersion || TTL || Endpoint_IPv4 || Endpoint_Port || Counter)`. Any substitution of the IP or port after signing then invalidates the signature, because those bytes are covered by it. This mirrors the general principle behind the “domain binding” and “sign the nonce inside the offer challenge” approaches used in adjacent 2025–2026 automotive-security proposals combining DNSSEC/DANE-style authentication with SOME/IP service discovery — the common thread across all of these designs is that the signature must be scoped to everything the receiver relies on to route traffic, not merely to the service identity.

The Counter field (§7) is included in the signed scope for the same reason: without it, a captured Offer with a still-valid signature and still-correct endpoint could be replayed verbatim at a later time (for example, to resurrect a decommissioned or compromised ECU's identity). Signing the counter along with the endpoint means a replayed message is only accepted once, within the freshness window defined next.

7. Replay and Freshness: From “Last-In-Wins” to “Last-Valid-In-Wins”

R2 diagnosed the routing vulnerability as a consequence of the gateway's “Last-In-Wins” design, which exists deliberately so a backup sensor can take over from a failed primary without a manual failover step. That property should not be removed — it is a legitimate safety feature — but it needs to apply only among senders who can prove they hold the legitimate signing key.

The proposed replacement rule is “Last-Valid-In-Wins”: the gateway (or a verification shim in front of it) tracks, per Service ID and Instance ID, the highest Counter value seen in a message with a valid signature. An incoming Offer is only allowed to update the routing table if its signature verifies against a known public key for that service and its Counter is strictly greater than the last accepted value. This keeps the self-healing failover property (a real backup ECU simply signs with its own valid key and a fresh, higher counter) while rejecting both forged offers (no valid key) and replayed offers (stale counter) — the two attack variants identified in §6.

The Counter itself can reuse the existing SOME/IP-SD Session ID field, which already increments monotonically per daemon instance, rather than inventing a new field — this keeps the design closer to “stateless” in spirit, since the gateway only needs to remember one integer per known service rather than a full session or handshake state.

8. Sequence Flow

The message exchange keeps the same three phases R1 and R2 already described (Initial Wait, Repetition, Main Offer cycle) and layers signing/verification onto the existing Offer transmission rather than adding new message types:

- Boot: the Camera ECU loads its Falcon-512 secret key (provisioned per §9) and computes one signature over (ServiceID, InstanceID, MajorVersion, TTL, its own Endpoint IP/Port, and the current Session ID/Counter) — see §11 for why this happens once per boot session rather than once per message.
- Initial Wait / Repetition: every cyclic OfferService broadcast attaches the PQ-Signature Option (§5) alongside the existing IPv4 Endpoint Option, exactly as camera_service.py already attaches its endpoint today — no new socket or transport path is required.
- Gateway verification: someipyd (or a thin verification layer wrapping it, see §9) parses the PQ-Signature Option, looks up the sender's public key in its local trust store by Service ID, recomputes the signed scope from the Entry and Endpoint Option it just received, and calls Falcon-512 verify. Only on success does it apply the “Last-Valid-In-Wins” update from §7.
- Dashboard subscription and first frame: unchanged from R2 — the Dashboard subscribes to the EventGroup once the gateway's routing table points at a verified endpoint, and TTFF is measured exactly as before, now inclusive of the added verification step.
- Attack scenario (the R3 acceptance test): attacker_service.py is extended to attempt the R2 hijack again — both a naive forged Offer (no valid signature) and an endpoint-substitution replay (§6) of a captured, validly-signed Offer with a swapped IPv4 Endpoint Option. Both must now be rejected by the gateway and never reach the Dashboard as a routing update.

9. Gateway Daemon Integration Plan

someipy's V2 architecture already centralizes all routing decisions in someipyd, which ECUs reach only through Unix Domain Sockets — this is the natural place to enforce verification, since it is the single component that currently implements “Last-In-Wins” and would need to implement “Last-Valid-In-Wins” instead. Two integration paths are worth prototyping and comparing during implementation:

- Native modification: patch someipyd itself to parse the PQ-Signature Option and apply the verification gate before updating its internal routing table. This is architecturally the cleanest option but requires modifying third-party daemon code.
- Verification shim: insert a small proxy process between the UDS socket and someipyd that intercepts OfferService messages, verifies them, and only forwards the ones that pass, leaving someipyd itself unmodified. This is more portable and easier to iterate on, at the cost of an extra hop.

Either path needs a small trust store: a mapping from Service ID to the corresponding Falcon-512 public key, and per §7, one “last accepted Counter” integer per (Service ID, Instance ID) pair. Both are small enough to hold in memory for the scale of a single vehicle's service catalog.

10. Public Key Provisioning

Sending the 897-byte Falcon-512 public key on every Service Discovery cycle would be wasteful and would also reopen a version of the MTU/overhead problem this design is trying to avoid, and it would let an attacker simply broadcast its own public key alongside a self-signed forged Offer, which is exactly the vulnerability signing is meant to close. Public keys are therefore treated as provisioning-time data, not Service Discovery data:

- Primary model: each ECU's Falcon-512 public key is injected at manufacturing or flash time (mirroring how a real OEM would provision a root-of-trust certificate) and distributed to the gateway's trust store out

of band — outside the SOME/IP-SD multicast channel entirely. For the testbed, this can be simulated with a simple local trust-store file read by someipyd/the verification shim at startup.

- Secondary model (for future certificate rotation): a separate, rarely-sent “Certificate Option” or a unicast request/response method could carry a public key or certificate on demand, but this is explicitly out of scope for the R3 prototype and is noted here only so it is not forgotten for R4.

11. Performance Budget and a Key Open Risk

R2's `crypto_test.py` measured Falcon-512 signing at ~2.43ms on Apple M4 (ARM64, desktop-class). That number is a reasonable proxy for a capable ARM automotive SoC, but it should not be assumed to hold on the low-end microcontrollers that actually run sensor ECUs like a backup camera. Published pqm4 benchmarks put Falcon-512 signing at roughly 19.6–22 million cycles on an ARM Cortex-M4 — at a typical automotive-grade Cortex-M4 clock (order of 100–200 MHz), that is on the order of 100–150ms, which alone would blow past the 50ms TTFF budget if a fresh signature had to be computed for every single cyclic Offer broadcast.

Verification is far cheaper and much less of a concern: pqm4 reports Falcon-512 verification at roughly 511,000 cycles on the same Cortex-M4 class hardware (on the order of a few milliseconds at typical clocks), and R1's equivalent ML-DSA measurement was already sub-millisecond on ARM64 desktop-class hardware, so verification sitting on the Dashboard's critical boot path is not expected to be the bottleneck.

This asymmetry motivates the “sign-once-per-boot-session” architecture referenced in §8: because the signed scope (§6) only changes when the Service ID, endpoint, or Counter changes — not on every 10ms cyclic broadcast — the Camera ECU can compute one signature at boot (or even before the vehicle's ignition/wake event, overlapped with other initialization) and reuse it for the entire Initial Wait and Repetition burst of that boot session, paying the expensive signing cost once rather than on the safety-critical path. A new signature is only required when the Counter needs to advance (e.g. after a genuine failover to a backup ECU), which is an infrequent event rather than a per-message one. This reframes signing cost as an amortized, off-path cost and leaves verification as the only real per-message cost on the boot-critical path — but this reframing is a design proposal, not yet a validated one, and is called out explicitly as the top experimental priority in §13.

12. Threat Model Update

- Endpoint substitution / signature replay onto a different port (§6): closed by signing the endpoint and counter, not just the service entry.
- Stale replay of a captured, validly-scoped Offer (§7): closed by the monotonic Counter check at the gateway.
- Downgrade attack: an attacker sends a plausible-looking but unsigned Offer, hoping a permissive gateway falls back to accepting it. The gateway's enforcement policy (§5.1) must default to “reject if PQ-Signature Option missing or invalid” for any service registered in its trust store, not “accept if absent.”
- Gateway/daemon as a new trust anchor: verification now happens centrally in someipyd (or its shim). Compromising that single process defeats the scheme for every service it routes, which is a form of the same single-point-of-failure concern R1's literature review raised about centralized DNS-based designs. This is accepted as a known limitation of a software-only R3 prototype and is flagged for R4, where a hardware security element could hold the verification logic and trust store outside the general-purpose OS process.
- Key compromise / rotation: out of scope for R3's prototype (§10), but the AlgID field in the PQ-Signature Option (§5) is reserved partly so a future key or algorithm rotation has somewhere to signal itself without a wire-format change.

13. R3 Experimental Plan

The following extends the existing R2 testbed (`someipyd`, `camera_service.py`, `dashboard_client.py`, `attacker_service.py`) rather than replacing it, so results remain comparable to the R2 baseline:

- Re-run `crypto_test.py` and log the Falcon-512 verify time to `R2_DATA.txt/R3_DATA.txt` — this number is currently missing from the retained dataset (§3) and is needed to validate the performance budget in §11.
- Implement PQ-Signature Option serialization/parsing (§5) and the signed-scope construction (§6) as a small shared module usable by both the camera and the verification layer.
- Implement the verification shim or native `someip` patch (§9) with the trust store and “Last-Valid-In-Wins” logic (§7).
- Re-run the R2 hijack scenario (`attacker_service.py`) unmodified against the new authenticated gateway and confirm the malicious-frame ratio drops from the ~66% baseline to 0%.
- Add the endpoint-substitution replay variant described in §8 as a second attack test, since it is not covered by R2's original attacker and is the specific gap this design closes.
- Measure the new end-to-end TTFF with verification in the loop, and separately measure/estimate the sign-once-per-session overhead (§11) to test whether it can be kept off the boot-critical path as proposed.
- Confirm empirically that a single authenticated Offer datagram (Entry + IPv4 Endpoint Option + PQ-Signature Option) stays under the ~1,472-byte usable payload with no IP-level fragmentation, validating §4's arithmetic on the wire rather than on paper.

14. Relation to Prior and Adjacent Work

This design keeps the application-layer, in-band approach R1 proposed (TLV options inside the existing SOME/IP-SD message) rather than the DNSSEC/DANE-style approach referenced in R1's bibliography [8], which secures discovery through an external DNS-like infrastructure. That family of designs has continued to develop — more recent proposals combine DNSSEC, DANE, and DANCE to authenticate in-vehicle service discovery using signed nonces inside challenge/offer exchanges, which independently supports this document's §6 conclusion that a signature must be bound to a fresh, endpoint-specific challenge rather than to service identity alone. The tradeoff remains the one R1 already identified: an external, infrastructure-based scheme avoids growing the SOME/IP-SD payload but reintroduces a dependency on an always-available directory service, which is the single-point-of-failure concern this project has deliberately designed around by keeping authentication data inside the SOME/IP-SD message itself.

15. R4 Preview

This document only covers design; R2's own plan already earmarks R4 for moving off the local-loopback software simulation and onto physical Hardware-in-the-Loop microcontrollers using optimized embedded libraries (e.g. PQM4 and the more recent 2025 Cortex-M4 Falcon optimizations) to replace the `pqm4`-derived estimates in §11 with directly measured cycle counts, and to validate the sign-once-per-session architecture against real automotive-grade clock speeds rather than the M4/desktop proxy used throughout R2 and this document.

16. Conclusion

R2 concluded that Falcon-512 is the right algorithm because it avoids the MTU collision outright. This document turns that conclusion into a specific wire format (a new PQ-Signature Option, §5), a specific signed scope that closes an endpoint-substitution gap neither R1 nor R2 had addressed (§6), a specific replay-resistance rule that generalizes R2's “Last-In-Wins” diagnosis into “Last-Valid-In-Wins” (§7), and a specific, falsifiable experimental plan (§13) built directly on the existing R2 testbed. The one open risk carried forward from this design phase is whether embedded-class signing latency (§11) can really be kept off the boot-critical path through session-level caching — that question is the first thing R3's implementation should settle.

Bibliography

- [1] AUTOSAR, “Specification of SOME/IP Service Discovery Protocol,” AUTOSAR Standard, Release 22-11 / 4.2.1.
- [2] AUTOSAR, “SOME/IP Protocol Specification,” AUTOSAR Foundation, Release 22-11.

- [3] D. Zelle, T. Lauser, D. Kern and C. Krauß, “Analyzing and Securing SOME/IP Automotive Services with Formal and Practical Methods,” Proc. 16th Int. Conf. on Availability, Reliability and Security (ARES), ACM, 2021.
- [4] C. Rathweg, C. Neumann and T. Pöppelmann, “Impacts of Post-Quantum Cryptography on Automotive Security: A Case Study,” ESCAR Europe, 2023.
- [5] National Institute of Standards and Technology (NIST), “FIPS 206 (Draft): FN-DSA (Falcon),” NIST Computer Security Resource Center, 2025.
- [6] T. Pornin, “Falcon on ARM Cortex-M4: an Update,” IACR ePrint 2025/123, 2025.
- [7] M. Kannwischer et al., “pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4,” Second PQC Standardization Conference, 2019.
- [8] M. Mueller, T. Häckel, P. Meyer and F. Korf, “Authenticated and Secure Automotive Service Discovery with DNSSEC and DANE,” IEEE Vehicular Networking Conference (VNC), 2023.
- [9] “Building Automotive Security on Internet Standards: An Integration of DNSSEC, DANE, and DANCE to Authenticate and Authorize In-Car Services,” arXiv:2506.13261, 2025.
- [10] IEEE Standard for Ethernet, “IEEE Std 802.3bw-2015 (100BASE-T1),” 2015.
- [11] A. Conti, S. Grottke and D. Gnedt, “PQ-CAN: A Framework for Simulating Post-Quantum Cryptography in Embedded Systems,” arXiv:2504.10730, 2025.